

```
%% SETUP: Extract training images (run once per session – /tmp/ is not
persistent)
% unzip('train_images_full.zip', '/tmp/retina_data_full')
```

%% SECTION 1: Load Data

```
labels = readtable('train_full.csv');
imageFolder = '/tmp/retina_data_full/train_images_full/';
files = fullfile(imageFolder, strcat(labels.id_code, '.jpg'));
grades = categorical(labels.diagnosis);
imds = imageDatastore(files, 'Labels', grades);
countEachLabel(imds)
```

ans = 5×2 table

	Label	Count
1	0	1805
2	1	370
3	2	999
4	3	193
5	4	295

%% SECTION 2: Train Test Split

```
rng(42);
[imdsTrain, imdsRest] = splitEachLabel(imds, 0.7, 'randomized');
[imdsVal, imdsTest] = splitEachLabel(imdsRest, 0.5, 'randomized');
```

%% SECTION 3: Class Weights

```
labelCounts = countEachLabel(imdsTrain);
classWeights = max(labelCounts.Count) ./ labelCounts.Count;
classWeightsVec = classWeights(:)';
disp(labelCounts)
```

Label	Count
0	1264
1	259
2	699
3	135
4	207

```
disp(classWeights)
```

```
1.0000
4.8803
1.8083
9.3630
6.1063
```

%% SECTION 4: Preprocessing

```
inputSize = [160 160 3];
```

```

augTrain = augmentedImageDatastore(inputSize, imdsTrain,
'ColorPreprocessing', 'gray2rgb');
augVal = augmentedImageDatastore(inputSize, imdsVal, 'ColorPreprocessing',
'gray2rgb');
augTest = augmentedImageDatastore(inputSize, imdsTest, 'ColorPreprocessing',
'gray2rgb');

%% SECTION 5: Load Network
net = imagePretrainedNetwork("efficientnetb0", NumClasses=5);

%% SECTION 6: Training Options
options = trainingOptions('adam', ...
    'InitialLearnRate', 1e-4, ...
    'MaxEpochs', 15, ...
    'MiniBatchSize', 32, ...
    'ValidationData', augVal, ...
    'ValidationFrequency', 20, ...
    'ExecutionEnvironment', 'cpu', ...
    'Plots', 'training-progress', ...
    'Verbose', true);

%% SECTION 7: Train Model (or load existing trained model)
if isfile('trainedNet_v3.mat')
    load('trainedNet_v3.mat')
    disp('Loaded existing trained model.')
else
    lossFcn = @(Y,T) crossentropy(Y, T, classWeightsVec, WeightsFormat="C");
    trainedNet = trainnet(augTrain, net, lossFcn, options);
    classNames = categories(imds.Labels);
    save('trainedNet_v3.mat', 'trainedNet', 'classNames', 'classWeights');
    disp('Model trained and saved successfully')
end

```

Iteration	Epoch	TimeElapsed	LearnRate	TrainingLoss	ValidationLoss
0	0	00:00:08	0.0001		4.3566
1	1	00:00:08	0.0001	6.4292	

```

%% SECTION 8: Evaluate – Default Threshold
scores = minibatchpredict(trainedNet, augTest);
[~, predictedIdx] = max(scores, [], 2);
classNames = categories(imds.Labels);
predictions = categorical(classNames(predictedIdx));
actual = imdsTest.Labels;
confusionchart(actual, predictions)

referableActual = double(actual) >= 3;
referablePred = double(predictions) >= 3;
TP = sum(referablePred==1 & referableActual==1);
FN = sum(referablePred==0 & referableActual==1);

```

```

TN = sum(referablePred==0 & referableActual==0);
FP = sum(referablePred==1 & referableActual==0);
sensitivity = TP/(TP+FN)
specificity = TN/(TN+FP)

%% SECTION 8B: Overall Accuracy
accuracy = sum(predictions == actual) / numel(actual);
fprintf('Overall 5-class accuracy: %.2f%%\n', accuracy*100);

for i = 1:5
    classIdx = (actual == classNames{i});
    classAcc = sum(predictions(classIdx) == actual(classIdx)) /
sum(classIdx);
    fprintf('Grade %s accuracy: %.2f%%\n', classNames{i}, classAcc*100);
end

%% SECTION 8C: ROC Curve for Referable DR Detection
referableProbTest = sum(scores(:, 3:5), 2);
actualBinaryTest = double(actual) >= 3;

[X, Y, T, AUC] = perfcurve(actualBinaryTest, referableProbTest, 1);

figure
plot(X, Y, 'LineWidth', 2)
hold on
plot([0 1], [0 1], 'k--')
xlabel('False Positive Rate (1 - Specificity)')
ylabel('True Positive Rate (Sensitivity)')
title(sprintf('ROC Curve for Referable DR Detection (AUC = %.3f)', AUC))
grid on
hold off

%% SECTION 9: Threshold Sweep (Validation Set)
valScores = minibatchpredict(trainedNet, augVal);
referableProbVal = sum(valScores(:, 3:5), 2);
actualBinaryVal = double(imdsVal.Labels) >= 3;

fprintf('--- Validation set threshold sweep ---\n');
for thresh = [0.2 0.3 0.4 0.5 0.6 0.7]
    predBinary = referableProbVal >= thresh;
    TP = sum(predBinary==1 & actualBinaryVal==1);
    FN = sum(predBinary==0 & actualBinaryVal==1);
    TN = sum(predBinary==0 & actualBinaryVal==0);
    FP = sum(predBinary==1 & actualBinaryVal==0);
    sens = TP/(TP+FN);
    spec = TN/(TN+FP);
    fprintf('Threshold %.2f: Sensitivity=%.3f, Specificity=%.3f\n', thresh,
sens, spec);
end

```

```

%% SECTION 10: Final Evaluation – Calibrated Threshold (0.40)
threshold = 0.40;
referableProbTest = sum(scores(:, 3:5), 2);
actualBinaryTest = double(actual) >= 3;
predBinaryTest = referableProbTest >= threshold;

TP = sum(predBinaryTest==1 & actualBinaryTest==1);
FN = sum(predBinaryTest==0 & actualBinaryTest==1);
TN = sum(predBinaryTest==0 & actualBinaryTest==0);
FP = sum(predBinaryTest==1 & actualBinaryTest==0);

sensitivity_final = TP/(TP+FN)
specificity_final = TN/(TN+FP)

%% SECTION 11: Single Image Prediction Demo
testImg = readimage(imdsTest, 1); % change index for different examples
testImg = imresize(testImg, [160 160]);
if size(testImg,3) == 1
    testImg = cat(3, testImg, testImg, testImg);
end
score = minibatchpredict(trainedNet, darray(single(testImg), 'SSCB'));
[~, predIdx] = max(score);
predictedGrade = classNames{predIdx};

figure
imshow(imresize(testImg, [300 300]))
title(['Predicted DR Grade: ' predictedGrade ' | True Grade: '
char(imdsTest.Labels(1))])

disp('Class probabilities:')
for i = 1:5
    fprintf('Grade %s: %.1f%%\n', classNames{i}, score(i)*100);
end

%% SECTION 12: Grid of 6 Random Predictions
figure
for i = 1:6
    idx = randi(numel(imdsTest.Files));
    img = readimage(imdsTest, idx);
    img = imresize(img, [160 160]);
    if size(img,3)==1, img = cat(3,img,img,img); end

    score = minibatchpredict(trainedNet, darray(single(img), 'SSCB'));
    [confScore, predIdx] = max(score);
    pred = classNames{predIdx};
    trueLabel = char(imdsTest.Labels(idx));

    subplot(2,3,i)
    imshow(img)

```

```

        title(sprintf('Pred: %s (%.0f%%) | True: %s', pred, confScore*100,
trueLabel))
end

%% SECTION 13: Grad-CAM Explainability
% Pick a test image to explain
idx = 1; % change this index to explain different images
img = readimage(imdsTest, idx);
img = imresize(img, [160 160]);
if size(img,3) == 1
    img = cat(3, img, img, img);
end
imgSingle = single(img);

% Get prediction
score = minibatchpredict(trainedNet, darray(imgSingle, 'SSCB'));
[~, predIdx] = max(score);
predictedGrade = classNames{predIdx};
trueLabel = char(imdsTest.Labels(idx));

% Compute Grad-CAM
scoreMap = gradCAM(trainedNet, imgSingle, predIdx);

% Overlay
figure
subplot(1,2,1)
imshow(img)
title(['Original | True: ' trueLabel])

subplot(1,2,2)
imshow(img)
hold on
imagesc(scoreMap, 'AlphaData', 0.5)
colormap jet
colorbar
title(['Grad-CAM | Predicted: ' predictedGrade])
hold off

%% SECTION 14: Grad-CAM Batch Grid (6 images)
figure('Position', [100 100 1400 800])

numImages = 6;
rng(1) % for reproducible image selection in the demo grid (not the data
split)
sampleIdx = randperm(numel(imdsTest.Files), numImages);

for i = 1:numImages
    idx = sampleIdx(i);
    img = readimage(imdsTest, idx);
    img = imresize(img, [160 160]);

```

```

if size(img,3) == 1
    img = cat(3, img, img, img);
end
imgSingle = single(img);

% Prediction + confidence
score = minibatchpredict(trainedNet, dldarray(imgSingle, 'SSCB'));
[confScore, predIdx] = max(score);
predictedGrade = classNames{predIdx};
trueLabel = char(imdsTest.Labels(idx));

% Grad-CAM
scoreMap = gradCAM(trainedNet, imgSingle, predIdx);

% Plot: original on top row, Grad-CAM overlay on bottom row
subplot(2, numImages, i)
imshow(img)
title(['True: ' trueLabel], 'FontSize', 9)

subplot(2, numImages, i + numImages)
imshow(img)
hold on
imagesc(scoreMap, 'AlphaData', 0.5)
colormap jet
hold off
title(sprintf('Pred: %s (%.0f%%)', predictedGrade, confScore*100),
'FontSize', 9)
end

sgtitle('Grad-CAM Explainability – Batch Sample', 'FontSize', 14,
'FontWeight', 'bold')

%% SECTION 15: Precision, Recall, F1 Score
precision = TP / (TP + FP);
recall = TP / (TP + FN);
f1_score = 2 * (precision * recall) / (precision + recall);

fprintf('Precision: %.4f\n', precision);
fprintf('Recall (Sensitivity): %.4f\n', recall);
fprintf('F1 Score: %.4f\n', f1_score);

%% SECTION 16: Metrics Summary Chart
metricNames =
categorical({'Accuracy', 'Sensitivity', 'Specificity', 'Precision', 'F1 Score'});
metricNames = reordercats(metricNames,
{'Accuracy', 'Sensitivity', 'Specificity', 'Precision', 'F1 Score'});
metricValues = [accuracy, sensitivity_final, specificity_final, precision,
f1_score];

```

```

figure
b = bar(metricNames, metricValues);
b.FaceColor = 'flat';
b.CData = [0.2 0.6 0.8; 0.3 0.7 0.4; 0.8 0.4 0.3; 0.6 0.4 0.8; 0.9 0.6 0.2];
ylim([0 1])
ylabel('Score')
title('Model Performance Summary – Referable DR Detection')
grid on

for i = 1:length(metricValues)
    text(i, metricValues(i) + 0.02, sprintf('%.1f%%', metricValues(i)*100),
        ...
        'HorizontalAlignment', 'center', 'FontWeight', 'bold')
end

```